

计算机系统概论（2025 秋）作业 3

班级：_____ 姓名：_____ 学号：_____

一、克拉茨猜想

克拉茨猜想（Collatz Conjecture）定义如下：

给定一个正整数 n ，若 n 为偶数，则下一项为 $n/2$ ；若 n 为奇数，则下一项为 $3n+1$ 。重复应用此规则，最终总会到达 1。

下面给出一个递归版本，它计算从 n 开始到 1 共经历了多少步（含最后一步）：

```
int collatz(int n) {  
    if (n == 1) {  
        return 1;  
    }  
    if (n % 2 == 0) {  
        return 1 + collatz(n / 2);  
    } else {  
        return 1 + collatz(3 * n + 1);  
    }  
}
```

编译后（Linux x86-64）得到如下汇编代码节选，请在空白处填入正确内容：

```
collatz:  
    cmpl    $1, _____  
    je      _____  
  
    movl    %edi, %eax  
    andl    $1, %eax  
    cmpl    $0, %eax  
    jne     _____  
  
.Leven:  
    movl    %edi, %eax  
    shrl    $1, %eax  
    _____  
    call    collatz  
    leal    1(%rax), %eax  
    jmp     .Lexit  
  
.Lodd:  
    leal    (%rdi, %rdi,2), %eax  
    addl    $1, %eax
```

```

    call    collatz
    leal    1(%rax), %eax
    jmp     .Lexit

.Lbase:
    movl    $1, %eax

.Lexit:
    ret

```

二、栈帧布局分析

给定 c 代码

```

void foo() {
    int m = 10;
    char buf[12];
    long x = 123456789;
    double y = 3.14;
    ...
}

```

编译器生成了如下 Linux x86-64 汇编片段（部分）：

```

foo:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $48, %rsp
    movl    $10, -4(%rbp)
    movq    $123456789, -24(%rbp)
    movsd   .LC0(%rip), %xmm0
    movsd   %xmm0, -32(%rbp)
    ...

```

请给出下面变量相对于 %rbp 的偏移

变量/寄存器

- m
- buf
- buf[5]
- x

- y

三、类与对象内存访问

给定 c++ 代码：

```
class Node {
public:
    int v1, v2;
    long t;
    Node(int a, int b, long c) {
        v1 = a;
        v2 = b;
        t = c;
    }
    long sum() {
        long x = v1;
        long y = v2;
        return x + y + t;
    }
};

int main() {
    Node nd(3, 4, 123);
    return nd.sum();
}
```

已知对应编译结果中（-O0）sum 函数入口汇编含有：

```
movq %rdi, -24(%rbp)
movq -24(%rbp), %rax
movl (%rax), %edx
movl 4(%rax), %ecx
movq 8(%rax), %rax
```

回答：

- this 指针在构造函数与 sum 中是如何传递的？
- sum 函数中如何根据 this 指针定位 v1/v2/t 的地址？

四、内存布局分析

程序中包含如下结构体：

```
struct Packet {
    T1 header;
```

```

    T2 flag;
    union {
        T3 u1;
        unsigned char buf[8];
    } u;
    struct Inner {
        T4 x;
        T5 y;
    } inner[2];
};

```

主函数执行了：

```

Packet p;
size_t sz = sizeof(p);
unsigned char* q = (unsigned char*)&p;
for (size_t i = 0; i < sz; i++) q[i] = 0xAA;

cout << p.header << endl;
cout << p.flag << endl;
cout << p.u.u1 << endl;
cout << p.inner[0].x << endl;
cout << p.inner[0].y << endl;

```

程序输出为：

```

sizeof(Packet) = 32
header = -1431655766
flag = 170
u1 = -6148914691236517206
inner[0].x = 2863311530
inner[0].y = -1431655766

```

已知：

- 0xAA = 170
- 0xAAAAAAAA = 2863311530
- 0xAAAAAAAAAAAAAAAAAAAA = 12297829382473034410 (unsigned) or -6148914691236517206 (signed)
- 0xAAAAAAAA interpreted as signed int = -1431655766

1. 请推导 Packet 中字段类型 T1, T2, T3, T4, T5
2. 下面给出某函数的节选汇编，假设 %rdi 传入的是指向 Packet 的指针，写出与这些汇编对应的 C 代码访问方式（即用 C 表达每条 mov 从 struct Packet 中取出的字段）。

```
movq    %rdi, -32(%rbp)
movq    -32(%rbp), %rax

movl    (%rax), %ecx
movq    8(%rax), %r8
movl    16(%rax), %r9d
movl    24(%rax), %r10d
```

五、虚拟内存映射

你正在为某嵌入式系统设计内存系统。该系统采用极简硬件，配置如下：

物理内存 8 MiB

每字节 8-bit，虚拟地址长度：32-bit

页大小：8 KiB

1. 基础参数推导

- (A) 若物理地址可完全覆盖物理内存，则其长度需为 _____ bit.
- (B) 页内偏移（PPO）的长度为 _____ bit.
- (C) 物理页号（PPN）长度为 _____ bit.
- (D) 为保证页表转换正确，虚拟页号（VPN）长度为 _____ bit.
- (E) 下列说法中正确的是（可多选）：

- a. PPO 与页大小有关
- b. VPN + VPO 的长度等于物理地址长度
- c. PPN 的长度由物理内存大小决定
- d. VPN 必须与物理页号等长

2. 地址翻译

系统运行时访问了虚拟地址：0x3FACEB00

已知：

- 页大小为 8 KiB
- 页表中该虚拟地址所属 VPN 对应的 PPN = 0x155

回答:

- (A) 该虚拟地址的 VPN = 0x_____
- (B) 页内偏移 VPO = 0x_____
- (C) 翻译后得到的物理地址 = 0x_____

3. TLB

该硬件的 TLB 总共 32 项, 4-way 组相联

- (A) TLB 有多少个 set? _____
- (B) TLBI (index) 长度: _____ bit
- (C) 若 VPN = 0x2F3B2, 则它的 TLBT (tag) = 0x_____

六、链接与重定位

下面是一段新的 C 程序 linktest.c, 经 gcc -c -O0 编译为 linktest.o, 并用于后续链接:

```
// linktest.c
extern int global_count;
extern void (*hook1)();

static void local_util() {}

void exported_func() {}

extern void ext_func();

void wrapper(void (*callback)()) {
    hook1();
    local_util();
    exported_func();
    ext_func();
    callback();
}
```

1. 函数调用的重定位

wrapper 中的五次调用分别是:

- hook1();
- local_util();
- exported_func();

- `ext_func()`;
- `callback()`;

请将它们分类到下列类型之一：

- 需要全局重定位（外部符号）
- 内部可解析（静态或本文件）
- 运行时才能确定（函数指针调用）

2. 需要以全局符号出现在 `.symtab` 中的有哪些？请列出 `linktest.o` 中必须以全局符号出现的符号及其原因。

3. 静态链接 vs. 动态链接

考虑两种情况：

- 情况 A：程序处于快速迭代开发中，运行时版本可能频繁更新
- 情况 B：程序用于实时系统（RTOS），要求极低延迟

问题：

在 A 和 B 中，你分别会选择静态链接还是动态链接？为什么？